E-Learning Analytics (FastAPI → Go Conversion)

This project provides a **Go implementation** of an e-learning analytics platform, converted from an original FastAPI-based design.
It includes route definitions, Redis key mapping, code scaffolding, and production-minded practices (DI, validation, caching, logging, testing).

---

1. High-level Mapping (FastAPI → Go / Gorilla Mux)

FastAPI Component	Go Equivalent
FastAPI app, routers, Pydantic models	`main.go`, Gorilla Mux routes, Go struct types, `go-playground/validator`
PyMongo (connection pooling)	`go.mongodb.org/mongo-driver` with context timeouts & pool options
redis-py	`github.com/redis/go-redis/v9` with connection pooling
Pydantic validation	Structs + `validate` tags + `validator` package
BackgroundTasks / APScheduler	Goroutines, channels, or `robfig/cron/v3`
JWT auth + Redis blacklist	`golang-jwt/jwt/v5` + middleware w/ Redis lookup
Dependency injection	Manual constructor injection, or `google/wire` if desired
Async/await	Goroutines + non-blocking Go Mongo/Redis drivers w/ contexts

---

2. Endpoints (FastAPI → Gorilla Mux)

Authentication

- `POST /auth/login` → issues JWT + Redis session
- `POST /auth/register`
- `POST /auth/refresh` → validate refresh token in Redis
- `DELETE /auth/logout` → blacklist JWT in Redis

Course Management

- `GET /courses` (cached, TTL 5m)
- `POST /courses` (instructor only, invalidates cache)
- `GET /courses/{id}` (cached)
- `GET /courses/{id}/analytics` (instructor only, cached)
- `PUT /courses/{id}/modules/{module_id}` (cache invalidation)

Student Progress

- `POST /progress/lessons/{lesson_id}/complete` (cache update)

- `GET /progress/dashboard` (heavily cached)
- `GET /progress/courses/{course_id}` (cached per user)

Analytics

- `GET /analytics/courses/{course_id}/performance` (TTL 15m)
- `GET /analytics/students/{student_id}/learning-patterns` (TTL 30m)
- `GET /analytics/platform/overview` (admin only, TTL 1h)

Cache Management

- `DELETE /cache/courses/{course_id}` (admin only)
- `GET /cache/stats` (admin only)

---

3. Redis Key Mapping

Key Pattern	Value	TTL
`user_session:{user_id}`	user_data	24h
`blacklisted_tokens:{token_jti}`	timestamp	token exp
`refresh_tokens:{user_id}`	token_data	7d
`course:{course_id}`	course_data	5m
`courses_list:{filters_hash}`	paginated results	2m
`progress:{user_id}:{course_id}`	progress_data	10m
`user_dashboard:{user_id}`	dashboard_data	5m
`analytics:course:{course_id}`	analytics_data	15m
`analytics:platform:overview`	platform_stats	1h
`search:{query_hash}`	search_results	30m
`popular_courses`	course_list	1h
`user_recommendations:{user_id}`	recommended_courses	6h

---

4. Project Structure

/cmd/server/main.go /internal/ /config /server /handlers /services /repositories /models /middleware /cache /workers # background workers / cron jobs /pkg/ /utils

---

5. Starter Code Snippets

Includes:

- `main.go` (Mongo + Redis + Gorilla Mux + DI setup)

- `models.go` (Users, Courses, Roles)
- Redis cache helpers
- `CourseService` (cache-aside example)
- Handlers (`GetCourse`)
- JWT middleware
- Repository interface (Mongo stub)

See examples in repo for details.

---

6. Cache Invalidation Patterns

- On **writes** (POST/PUT/DELETE to a course):
  Invalidate `course:{id}`, `courses_list:{filters_hash}`, `popular_courses`.
  Optionally enqueue a background cache-warm job.
- On **progress updates**:
  Update `progress:{user_id}:{course_id}` and `user_dashboard:{user_id}`.
- On **logout**:
  Add token to `blacklisted_tokens:{jti}` with expiry = token lifetime.

---

7. Background Tasks

- Use goroutines or worker pools for async tasks (cache warming, analytics recompute).
- Use `robfig/cron/v3` for scheduled tasks (e.g., rebuild `analytics:platform:overview` hourly).

---

8. Validation & Error Handling

- Use `go-playground/validator` for request payload validation.
- Return consistent JSON error format:

```
{"error":"message", "code":1234, "details": {...}}
```

9. Logging & Metrics

- Use **Logrus** or **Zap** for structured logging.
- Expose `/metrics` (Prometheus) for:
  - Cache hit/miss counters
  - DB latency
  - Request latencies
- visualize the metrics with **Grafana**

---

10. Tests

- **Unit tests**: mock repositories & cache.
- **Integration tests**: use testcontainers or in-memory Redis + Mongo.
- Explicitly test:
  - Cache hit/miss behavior
  - TTL expirations

Route Registration Example

```
r.HandleFunc("/auth/register", h.Register).Methods("POST")
r.HandleFunc("/auth/refresh", h.Refresh).Methods("POST")
r.HandleFunc("/auth/logout", h.Logout).Methods("DELETE")

r.HandleFunc("/courses", h.ListCourses).Methods("GET")
r.HandleFunc("/courses", h.CreateCourse).Methods("POST")
r.HandleFunc("/courses/{id}", h.GetCourse).Methods("GET")
r.HandleFunc("/courses/{id}/analytics",
h.GetCourseAnalytics).Methods("GET")
r.HandleFunc("/courses/{id}/modules/{module_id}",
h.UpdateModule).Methods("PUT")
```

12. Run Notes / Quick Checklist

- **MongoDB**: set proper indexes (course ID, user ID, created_at).
- **Redis**: configure max memory, eviction policy, TLS in production.
- **JWT**: secure signing key, include `jti` claim for revocation.
- **Environment**: load config from env vars (`DB_URI`, `REDIS_URL`, JWT keys).
- **Containerization**: Dockerfile + docker-compose for local Mongo + Redis.

---

13. Exclusions (on purpose)

- Full CRUD for all nested structures (modules/lessons/quizzes).
- Complete Mongo repository implementations.
- UI, migrations, and full test coverage.

Patterns provided should make these extensions straightforward.

---

14. Next Steps

Possible extensions:

- Full handler implementations for all endpoints.
- Repository implementations with Mongo aggregation examples.
- Auth service (JWT + Redis-backed refresh/blacklist).
- Background worker + cron job for analytics recompute.
- End-to-end example (register → login → create course → get course).

15. Deployment & Networking (Optional)

- **Nginx reverse proxy** in front of the Go service:
  - TLS termination
  - Reverse proxy to `server:8080`
  - Rate limiting / request logging
- **Distributed architecture simulation** (with Docker Compose):
  - `auth-service` (JWT + sessions)
  - `analytics-service` (course stats, background workers)
  - `api-gateway` (Nginx)
  - Shared Mongo + Redis
- Local setup:
  - `docker-compose up --build`
  - Nginx available at `http://localhost:80`, proxies to services